Self-supervised pretraining and label efficiency

Scientific question: when many wake fields are available but only a few of them are labelled for the case we care about, does pretraining a network on the unlabelled fields reduce the number of labels the new case needs, and does the pretrained network beat a classical method that receives exactly the same information?

The lecture reproduces, on the FlowMLLab lattice-Boltzmann cylinder wake, the protocol of MAPA (Tang, Spalding and Cogan, Duke University, 2026), which pretrains a masked autoencoder on unlabelled intracranial recordings and then measures decoding error against the number of labelled trials of a new subject. Our unlabelled data are two wake trajectories (Re = 90 and 100), our new subject is the Re = 110 trajectory, our label is a complete velocity frame, and our classical baseline is gappy POD (Everson and Sirovich, 1995). Companion notebook: `notebooks/week07_3/W7_3_Masked_Pretraining_Label_Efficiency.ipynb`; module: `flowmllab/masked_pretraining.py`.

Prerequisites: POD and least squares (Week 5), the wake data set (Week 7), validation-only selection and the persistence-above-100% argument (Week 7.2). Runtime is about ten minutes on two CPU cores, or three minutes with the notebook's QUICK switch.

Definitions before use

- **Label.** A complete 32 x 78 frame of the transverse velocity v/U of the target trajectory. "Given k labelled frames" means the method may read k complete frames of Re = 110 and nothing else about that trajectory.
- **Unlabelled data.** Complete frames of other trajectories (Re = 90 and Re = 100). Nothing about Re = 110 is in them. Self-supervised methods manufacture their own training signal from such data.
- **Patch and mask.** A patch is a 4 x 6 block of pixels; the window is an 8 x 13 grid of 104 patches. A mask hides a random 75% of the patches (78 of 104). The 26 visible patches are the input; the hidden ones are the target.
- **Masked autoencoder (MAE).** A transformer encoder that reads only the visible patches, and a small decoder that fills in the hidden ones. The loss is the mean squared error on hidden patches only (He et al., 2022).
- **Self-supervised pretraining.** Training the MAE on unlabelled frames with fresh random masks. No label is used: the frames supervise themselves.
- **Zero-shot.** Applying a pretrained model, or a POD basis fitted on other trajectories, to the target with no labelled frame.
- **Linear probe.** Freezing the pretrained encoder and fitting only a ridge-regression readout from its features to the full frame on the k labelled frames.
- **Fine-tuning.** Continuing to train all weights of the pretrained model on the k labelled frames.
- **From scratch.** The same architecture from random weights, trained on the k labelled frames with the same downstream budget as fine-tuning.
- **Gappy POD.** Fit a POD basis to complete frames; for a masked frame, solve a least-squares problem for the modal coefficients from the visible pixels only; reconstruct the hidden pixels from the basis.
- **Matched baseline.** A baseline that sees the same frames and the same visible pixels as the neural method. Without it, an improvement cannot be attributed to the method rather than to the information it received.
- **Label-efficiency curve and label saving.** Test error against k. The label saving of a pretrained method is the number of labels the reference method needs to reach the error the pretrained method reaches with its smallest k.
- **Relative L2 error.** 100 x ||prediction - truth|| / ||truth||, reported over the hidden pixels only. Every method first copies the visible pixels into its output, so the full-frame error is always smaller than the hidden-pixel error.

Gappy POD: the classical answer to the completion task

Write a complete frame as a vector q with 2496 entries and a POD basis fitted on complete frames as the mean q_mean and the modes Phi (2496 x r). For a masked frame let V be the index set of visible pixels. Gappy POD solves the least-squares problem

```
minimise over a: || Phi_V a - (q_V - q_mean,V) ||^2 + eps ||a||^2
reconstruction: q_hat = q_mean + Phi a
```

where Phi_V holds the visible rows of the modes and eps is a tiny ridge for numerical safety. With 624 visible pixels and r at most 64 the system is overdetermined ten times over, so the coefficients are a projection, not an interpolation, and the hidden pixels follow from the subspace. The rank r is the only choice; it is selected on the validation trajectory Re = 105. A larger rank represents more of the field but makes the visible-pixel system worse conditioned, so the validation curve has a minimum.

Worked example with one mode. Suppose the field is q = q_mean + a phi with a single mode phi, and three pixels are visible with mode entries phi_V = (0.6, 0.8, 0.0) and centred observations (1.2, 1.6, 0.3). The normal equation is (0.36 + 0.64 + 0) a = 0.6 x 1.2 + 0.8 x 1.6 + 0 x 0.3, so a = 2.0. The third pixel, where the mode vanishes, contributes nothing to the fit but its residual 0.3 is the part of the observation that the subspace cannot explain; it is exactly this residual that a rank chosen too small leaves in every hidden pixel. With the basis in hand the hidden pixels are q_mean + 2.0 phi at their locations.

Three bases are used. The transferred basis is fitted on the 562 unlabelled frames (Re = 90 and 100) and needs no label of the target. The target-only basis is fitted on the k labelled frames, so its rank is at most k - 1 and k = 1 gives the mean field. The pooled basis is fitted on the unlabelled frames plus the k labelled ones; it is the classical analogue of fine-tuning.

The masked autoencoder and its training

Each visible patch (24 pixels) is embedded linearly into a 64-dimensional token and given a learned position embedding. A two-layer transformer encoder (4 heads, GELU, pre-norm) attends over the 26 visible tokens only; it never sees a hidden patch and cannot learn to copy. A one-layer decoder of width 32 receives the encoded visible tokens and a shared learned mask token at every hidden position, each with its own position embedding, and a linear head predicts the 24 pixels of every patch. The model has about 90 thousand parameters.

Training uses AdamW (weight decay 0.05), a cosine learning-rate schedule from 2e-3, batches of 8 frames and a fresh random mask for every frame at every step, so 562 frames yield an unlimited supply of completion problems. The masked loss on Re = 105 with fixed masks is evaluated before training and once per pass over the data, and the weights with the lowest validation loss are restored: early stopping selects on validation, never on test, and the starting weights are a candidate, so fine-tuning that only hurts is rejected.

The loss curve has a plateau. For the first few hundred steps the network outputs the mean field, so the masked loss equals the variance of v (about 0.0625 in (v/U)^2). Then it breaks through and falls by two orders of magnitude. A run stopped inside the plateau looks like a failed method; so does a from-scratch model given only a few hundred steps, which is why the protocol adds a budget control.

Why the encoder sees only visible patches. If hidden positions entered the encoder as zeros, the network could learn that "zero means hidden" and use the pattern of zeros; excluding them forces the representation to be built from the observed physics alone, and it makes the encoder four times cheaper. The decoder, not the encoder, is where the mask token lives.

Frozen features for the probe. The encoder output contains 64 features for each visible patch. The probe averages only those visible tokens, producing one 64-component representation whose size is independent of the mask, while its values depend on visible content, then maps it to the 2496 pixels of the full frame. With k labelled frames and 8 random masks per frame it is fitted by ridge regression, and the ridge strength is chosen on Re = 105. This visible-token-pooled pooling follows the transfer-learning logic of MAPA more closely than flattening a 104 x 64 array padded with zeros at hidden positions.

The frozen protocol

Trajectories and roles: Re = 90 and 100 are unlabelled pretraining data; Re = 105 is the validation trajectory for every selection (POD rank, ridge strength, early stopping); Re = 110 is the target. Its frames [0, 160) form the labelled pool and frames [210, 281) are the test window, scored with fixed random masks and never fitted; frames [160, 210) separate the two, as in Week 7.2. Three mask seeds change the test masks and the k frames drawn from the pool (evenly spread over the pool for k up to 8, random beyond). For k in {1, 2, 4, 8, 16, 32, 64, 128} every method is refitted:

- `pod_target`: gappy POD with the target-only basis (rank at most k - 1, chosen on Re = 105).
- `pod_pooled`: gappy POD with the pooled basis (rank chosen on Re = 105).
- `mae_probe`: frozen encoder plus ridge readout, 8 random masks per labelled frame.
- `mae_scratch`: the architecture from random weights, 300 AdamW steps with fresh masks, early stopping on Re = 105.
- `mae_finetune`: the pretrained model, the same 300 steps at learning rate 5e-4, early stopping on Re = 105.

Protocol attribution: the 300-step scratch and fine-tuning pipelines share labelled frames and architecture but use learning rates 2e-3 versus 5e-4 and mask seeds 400+seed versus 500+seed. Their difference is not an isolated initialization effect. A matched-rate, matched-mask ablation would be a separate validation-selected study.

Zero-shot entries: gappy POD with the transferred basis and the pretrained MAE. Budget control: the architecture from scratch on the seed-0 frames at k = 2, 8 and 128 for the full pretraining budget of 5000 steps. All errors are hidden-pixel relative L2 averaged over the 71 test frames; the tables report mean and sample SD over the three seeds, which are repeated draws on one CFD trajectory and not independent flows.

Retained masked-pixel relative L2 error (%) on the Re = 110 test window, mean +/- sample SD over 3 mask seeds

k	POD target-only	POD pooled	MAE probe	MAE scratch	MAE fine-tuned
1	141.36 +/- 5.47	1.36 +/- 0.04	141.36 +/- 5.47	114.74 +/- 19.71	17.62 +/- 0.13
2	76.11 +/- 4.15	1.30 +/- 0.01	75.27 +/- 3.75	99.86 +/- 0.38	17.62 +/- 0.13
4	24.85 +/- 0.25	1.15 +/- 0.03	22.94 +/- 0.62	99.39 +/- 0.58	10.03 +/- 0.38
8	5.30 +/- 0.08	0.90 +/- 0.03	13.65 +/- 0.93	99.78 +/- 0.16	7.95 +/- 0.38
16	5.14 +/- 6.18	0.71 +/- 0.11	16.12 +/- 7.15	100.15 +/- 0.91	8.51 +/- 0.82
32	0.61 +/- 0.14	0.28 +/- 0.04	9.12 +/- 0.56	100.33 +/- 0.27	7.79 +/- 0.51
64	0.04 +/- 0.00	0.18 +/- 0.02	8.11 +/- 0.91	99.99 +/- 0.27	7.67 +/- 0.25
128	0.02 +/- 0.00	0.15 +/- 0.01	7.76 +/- 0.96	99.72 +/- 0.28	7.51 +/- 0.34

POD target-only: gappy POD with a basis from the k labelled frames; POD pooled: basis from the unlabelled frames plus the k labelled ones; MAE probe: frozen encoder with a ridge readout; MAE scratch: the architecture from random weights, 300 steps; MAE fine-tuned: the pretrained model, 300 steps.

Zero-shot rows (no labelled frame): transferred gappy POD 1.40%, pretrained MAE 17.62%. Budget control (seed 0, the architecture trained from scratch for the full pretraining budget of 5000 steps on the same labelled frames): k = 2: 94.37% from scratch against 17.52% fine-tuned; k = 8: 8.67% from scratch against 8.37% fine-tuned; k = 128: 6.29% from scratch against 7.91% fine-tuned. Pretraining took 63 s and the label-efficiency loop 196 s on 2 CPU threads.

Reading the retained numbers

Three findings, each bounded by the data it was measured on.

Pretraining helps the network, most where labels are fewest. With identical labelled frames and identical downstream update count, the fine-tuned model is never worse than its zero-shot starting point (early stopping on Re = 105 may keep

the pretrained weights), improves on it from k = 4 (about 10%) and settles near 8% from k = 8 upwards, while the from-scratch model never leaves the mean-field plateau (about 100%) at any k. The budget control addresses the objection that it only needed more steps: with the full pretraining budget on the target frames the from-scratch model still fails at k = 2, comes within about a percentage point at k = 8, and at k = 128 is at least as good as the 300-step fine-tuned model. This MAPA-inspired teaching comparison on a wake shows that, for these two pipelines, the benefit of pretraining is concentrated in the low-label regime, and with enough labels and compute the unlabelled trajectories add nothing on this data.

The classical method wins. Gappy POD with the transferred basis and zero labels is already an order of magnitude more accurate than the fine-tuned network, and the pooled basis improves with every labelled frame down to a fraction of a percent. The wake at Re = 90 to 110 is periodic and low-rank: 64 modes fitted on the other Reynolds numbers span the target field almost exactly, and 624 visible pixels overdetermine the coefficients. A transformer with 90 thousand weights and 562 training frames cannot beat a subspace that is nearly the truth. The target-only basis, by contrast, is poor for k below 8 because a rank of k - 1 cannot represent the wake, and from k = 64 it is the most accurate method of all: labels are worth more to POD than to the network on this data.

The frozen probe is weaker than the pretrained decoder at very small k. The visible-token-pooled 64-component probe is much cheaper and improves rapidly as labels are added, reaching about 8% at k = 64 to 128. At k = 1 to 4, however, a linear map from very few augmented examples to 2496 output pixels is underdetermined, whereas the pretrained decoder already contains the nonlinear completion map. Probing measures the linear accessibility of a representation, not the best use of the pretrained network.

When does the recipe pay off? When the data are not low-rank (turbulence, experiments with noise, several regimes in one data set), when no matched linear baseline exists, or when the downstream label is not the field itself, as in MAPA, which decodes behaviour and speech from the recordings rather than reconstructing them. The exercises move this experiment towards those conditions instead of declaring a winner on this one.

Limits of the evidence: educational LBM at 12 lattice nodes per diameter, not grid-independent; four trajectories at nearly the same Reynolds number; the target trajectory was inspected in Weeks 5, 7 and 7.2, so this is a protocol demonstration and not a blind benchmark; three mask seeds of one trajectory; CPU-sized models and budgets.

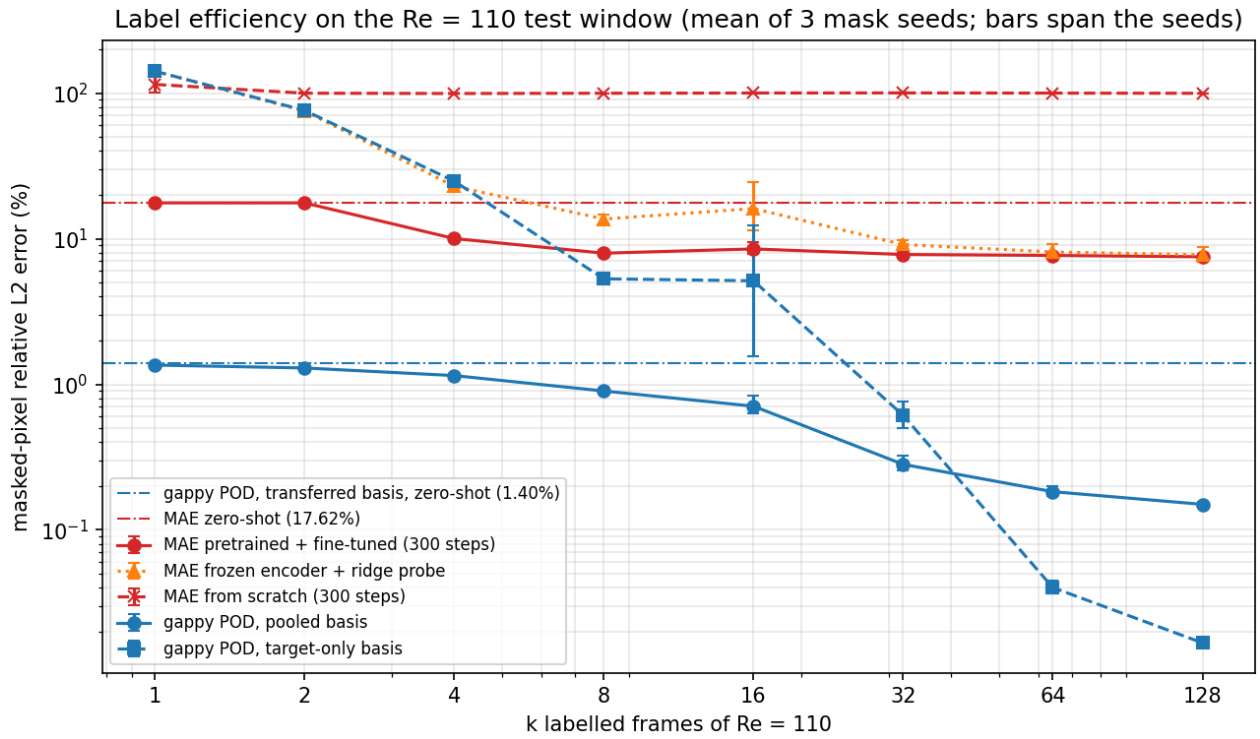


Figure 1. Masked-pixel error against the number of labelled Re = 110 frames, mean over three mask seeds of one trajectory; the bars span the three seeds. Horizontal lines are the zero-shot methods.

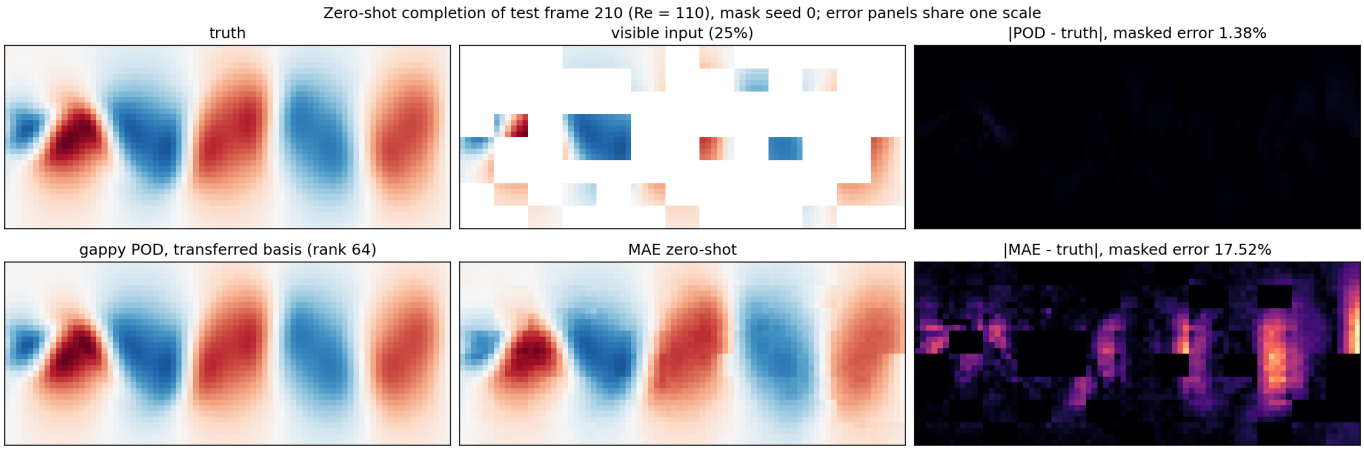


Figure 2. Zero-shot completion of test frame 210 (mask seed 0): truth, the 25% of pixels every method receives, and the two reconstructions with absolute-error maps on one scale.

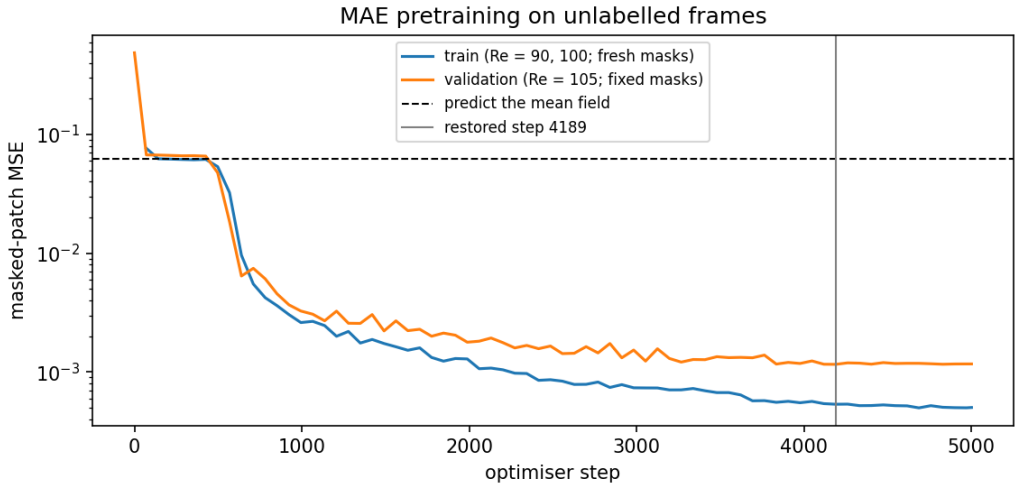


Figure 3. Masked-patch MSE during pretraining. The dashed line is the variance of v, the loss of predicting the mean field; the network sits on that plateau for a few hundred steps before it learns.

Reproducibility

The notebook is the single source of the retained evidence. `python qa/build_week07_3_materials.py --execute` rebuilds the notebook text, executes it with `FLOWMLLAB_WRITE_RESULTS=1` so that `results/week07_3_pretraining/` is refreshed (metrics.json with the protocol, every record, the pretraining history, timings, environment and data hashes; the three figures), and rebuilds this PDF from the retained metrics. A plain Run All, in Colab or locally, writes to a fresh temporary folder, compares its numbers with the retained ones and asserts that the tracked wake data are unchanged. `tests/test_masked_pretraining.py` checks the patch layout, the masks, gappy POD on exact low-rank data, the ridge readout, the frame sampling, the label-saving arithmetic and, when PyTorch is installed, that the MAE learns a small completion task and that its encoder never receives a hidden patch.

Exercises

1. Structured masks. Replace the random patch mask by a contiguous hidden block (for example the downstream half of the window). Which method degrades more, and how does the least-squares argument change when the visible pixels no longer sample every mode?
2. A harder downstream label. Use the masked v field as input and the full vorticity field (also in the archives) as the label. Build the matched classical baseline (a linear map from gappy-POD coefficients of v to vorticity, fitted on the k frames) and redraw the curve.
3. Noise. Add Gaussian noise of 10% of the field RMS to the visible pixels, as in Week 7.2. Reselect the POD rank and the ridge strength on Re = 105 and report which method is more robust.
4. Budget fairness. Give the from-scratch model 1000 and 3000 steps at every k and plot the family of curves. At what budget does it catch the zero-shot model, and does it ever catch the fine-tuned one below k = 16?

5. Falsification. Write down, before running anything, the observation that would make you conclude that pretraining does not help on this data. Then check whether the retained evidence contains it.

References

K. He, X. Chen, S. Xie, Y. Li, P. Dollar and R. Girshick, Masked autoencoders are scalable vision learners, CVPR 2022, doi:10.1109/CVPR52688.2022.01553.

B. Tang, T. Spalding and E. Cogan, Pretraining for sample-efficient neural interfaces (MAPA), Duke University, arXiv:2609.13507, 2026, https://bentang18.github.io/mapa-page/.

R. Everson and L. Sirovich, Karhunen-Loeve procedure for gappy data, Journal of the Optical Society of America A 12, 1657-1664, 1995, doi:10.1364/JOSAA.12.001657.

M. McCabe et al., Multiple physics pretraining for physical surrogate models, arXiv:2310.02994, 2023. M. Herde et al., Poseidon: efficient foundation models for PDEs, NeurIPS 2024, arXiv:2405.19101.

Data: FlowMLLab cylinder-cfd-v1 (`data/modal_labs`). The module, notebook and figures are independently authored for this repository; no MAPA code or data is used.